

Introduction

`mlr3` is the modern R6-based successor to `mlr`, a feature-complete machine-learning framework that abstracts around five core objects: Tasks (data + target + roles), Learners (algorithms), Resamplings (CV schemes), Tuners (search strategies), and Benchmarks (multi-learner comparisons). Like `tidymodels`, `mlr3` provides a unified interface across dozens of underlying engines; unlike `tidymodels`, it uses R6 object-oriented classes rather than tibbles, giving more flexibility but a steeper learning curve.

Prerequisites

A working understanding of R6 classes, machine-learning pipeline concepts, and cross-validation-based hyperparameter tuning.

Theory

The core `mlr3` workflow:

1. **Task**: wraps data with target and feature-role specifications.
2. **Learner**: an algorithm (e.g., `lrn("classif.ranger")` for random forest classification).
3. **Resampling**: a cross-validation scheme (`rsmp("cv", folds = 5)` for 5-fold CV).
4. **Tuner with AutoTuner**: wraps a learner plus a search space, inner resampling, and search strategy into a self-tuning learner.
5. **Outer resampling**: evaluate the AutoTuner on outer folds for honest nested-CV performance estimation.

`mlr3pipelines` extends the framework with preprocessing graphs and meta-learners (stacking, bagging) as composable nodes — equivalent in role to `tidymodels` `recipes` and `workflows`.

Assumptions

Task roles (target, predictors, stratification variables) are correctly specified; hyperparameter search spaces cover plausible ranges; resampling matches the inferential question (random for IID, blocked for time series, spatial for geography).

R Implementation

```
library(mlr3); library(mlr3learners); library(mlr3tuning)

set.seed(2026)
df <- data.frame(
  x1 = rnorm(500), x2 = rnorm(500),
  x3 = rnorm(500), x4 = rnorm(500),
  y = factor(sample(c("yes", "no"), 500, replace = TRUE))
)

task <- as_task_classif(df, target = "y")

# Tunable random forest
lrn_rf <- lrn("classif.ranger",
  mtry = to_tune(1, 4),
  min.node.size = to_tune(5, 20),
  num.trees = 300)

at <- AutoTuner$new(
  learner = lrn_rf,
```

```

  resampling = rsmp("cv", folds = 3),
  measure     = msr("classif.ce"),
  search_space = NULL,
  terminator  = trm("evals", n_evals = 10),
  tuner       = tnr("random_search")
)

outer <- rsmp("cv", folds = 3)
rr <- resample(task, at, outer, store_models = TRUE)
rr$aggregate(msr("classif.ce"))

```

Output & Results

AutoTuner performs hyperparameter tuning on inner resampling folds and selects the best configuration. `resample()` evaluates the AutoTuner on outer folds, producing nested-CV estimates of generalisation performance. `rr$aggregate()` returns the chosen metric (classification error here) averaged across outer folds with bootstrap-based standard errors.

Interpretation

A reporting sentence: “Nested 3×3 cross-validation with mlr3 AutoTuner (10-evaluation random-search inner tuning of random forest hyperparameters) estimated classification error of 0.25 ± 0.02 for the tuned random forest; the nested wrapping prevents the optimistic bias of tuning and evaluating on the same folds.” Always state the outer/inner CV scheme and the search budget.

Practical Tips

- AutoTuner is the right pattern for honest nested cross-validation; tuning and evaluating on the same resampling produces optimistically biased estimates.
- `mlr3pipelines` formalises preprocessing as pipeline operations (PipeOps); the equivalent of tidymodels `recipes`. Compose nodes into Graphs for complex pre/post-processing.
- `mlr3tuningspaces` provides sensible default search spaces per learner; useful when you don’t know the right ranges.
- `benchmark_grid()` compares multiple learners on the same task under identical resampling, giving fair head-to-head performance comparisons.
- For big data, use `mlr3spatiotempcv` or `mlr3temporal` for time-aware resamplings; standard random CV leaks information for time-series problems.
- Parallelise tuning and resampling with `future::plan(multisession)`; mlr3 respects the `future` framework.

R Packages Used

`mlr3` for the core framework; `mlr3learners` for the curated learner library; `mlr3tuning` and `mlr3tuningspaces` for hyperparameter optimisation; `mlr3pipelines` for graph-based pipelines; `mlr3viz` for visualisation; `mlr3spatiotempcv` and `mlr3temporal` for time- and space-aware resampling.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren’t.
- Red flags in tables and figures.
- What to verify.

- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis